W4A16 INT4 Weight-Only GEMM in Triton: An EECS 199 Report

Qiwen Xiao
qiwenx6@uci.edu

***Abstract*—This report summarizes my progress this quarter on W4A16 INT4 weight-only GEMM in Triton, with emphasis on decode-fused execution. I implemented four relevant pieces: (1) an affine per-group INT4 quantizer for weight-only compression, (2) a naive baseline that dequantizes weights and then calls dense matrix multiplication, (3) an autotuned Triton fused data-parallel kernel used as the paper-style baseline, and (4) a custom Triton implementation that supports either a single-pass store path or a split-K path implemented as partial accumulation followed by a separate reduction kernel. The main empirical result is that split-K is clearly beneficial in the decode regime when M is very small, especially at split-K $= 4$, but that this benefit weakens as N grows and disappears once the workload moves toward prefill. In prefill, the autotuned fused data-parallel kernel is the best Triton variant, and for sufficiently large M the naive dequantize-then-matmul baseline overtakes all current fused Triton kernels.**

I. Project Scope

The objective of this quarter's work was to investigate fused W4A16 matrix multiplication for large language model (LLM) inference in Triton, with particular emphasis on the distinct computational characteristics of the decode and prefill phases. In the implementation studied in this repository, the activation matrix $A \in \mathbb{R}^{M\times K}$ is stored in FP16, whereas the weight matrix $W \in \mathbb{R}^{K\times N}$ is represented using weight-only INT4 quantization with per-group affine parameters. The primary goal was not merely to confirm the general benefit of fusion, but rather to determine which Triton execution strategy performs best under different matrix shapes.

More specifically, the study compares three fused execution strategies. The first is a **fused data-parallel baseline** without split-K. The second is a **fixed-tile fused store kernel** with split-K fixed to 1. The third is a **split-K implementation** that avoids atomic accumulation by writing FP32 partial sums and performing the reduction in a separate kernel. These strategies were evaluated across two representative inference regimes. The decode regime corresponds to very small batch dimensions, with $M \in \{1, 2, 4, 8, 16, 32\}$, where output-channel parallelism may be limited and split-K can potentially expose additional concurrency. The prefill regime corresponds to larger batch dimensions, with $M \in \{128, 256, 512, 1024\}$, where conventional data-parallel tiling is expected to provide sufficient parallel work.

II. Implementation Summary

A. Quantization Format

The repository implements affine weight-only INT4 quantization with group size 128. For each group of 128 rows and each output column, the quantized representation stores an FP16 scale together with a UINT8 zero point. Quantization is defined as

$$q = \mathrm{clamp}\Big(\mathrm{round}\Big(\frac{x}{s} + z\Big), 0, 15\Big),$$

while dequantization is given by

$$\hat{x} = (q - z)s.$$

Because two INT4 values are packed into a single UINT8, the packed weight tensor has shape $[K, \lceil N/2 \rceil]$. This format reduces memory traffic while preserving the information required for on-the-fly dequantization inside the fused kernels.

B. Compared Kernels

The benchmark script evaluates four implementations. The naive baseline first dequantizes the full packed weight matrix to FP16 and then performs the matrix multiplication $A@W$ using PyTorch. The data-parallel fused baseline performs dequantization and GEMM jointly in Triton using a two-dimensional data-parallel launch grid together with Triton autotuning; this serves as the main reference implementation in the repository. In addition, the study includes a custom fused store kernel with fixed tile sizes and direct FP16 output stores, corresponding to split-K $= 1$. Finally, a custom split-K kernel is evaluated for split-K values of 2 and 4. Rather than relying on atomic accumulation, this kernel writes FP32 partial results into an intermediate tensor $P[\mathrm{split_k}, M, N]$ and then invokes a second kernel to reduce these partial sums to FP16 output.

Method	Split-K	Description
Naive baseline	none	Dequantize the full packed weight matrix to FP16, then compute $A@W$ with PyTorch.
DP fused baseline	none	Triton implementation that fuses dequantization and GEMM using a two-dimensional data-parallel grid and Triton autotuning.
Store kernel	1	Custom Triton fused kernel with fixed tile sizes and direct FP16 output stores.
Split-K kernel	2 or 4	Custom Triton kernel that writes FP32 partial tiles into $P[\text{split_k}, M, N]$ and reduces them to FP16 output in a second kernel.

TABLE I: Methods compared in the repository benchmarks.

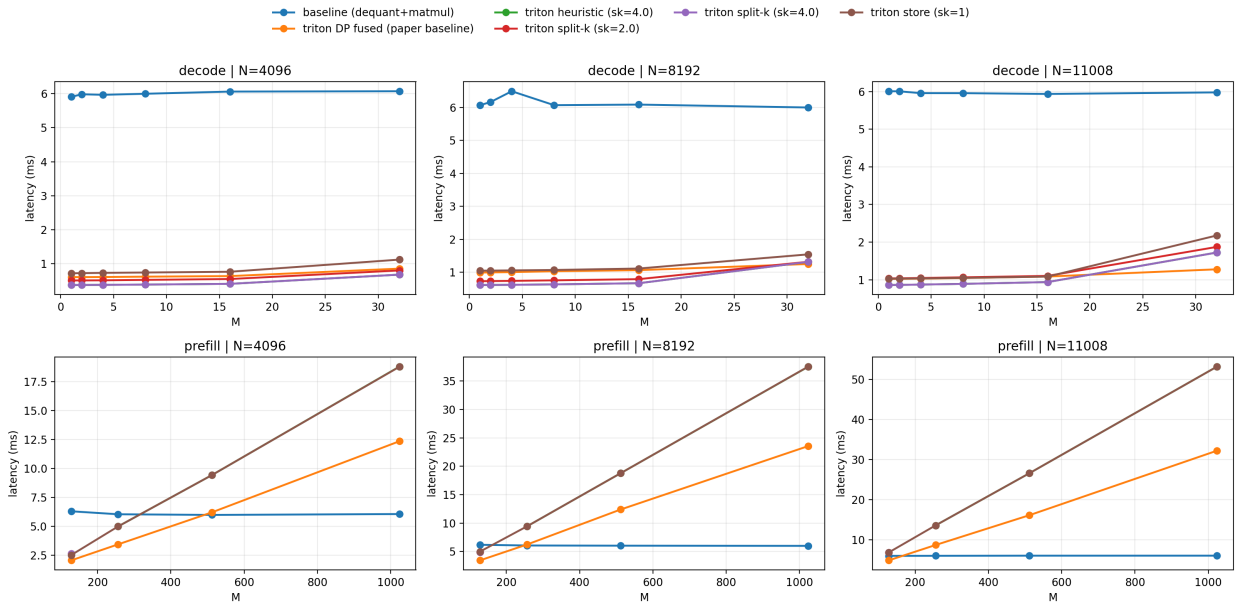

Fig. 1: Absolute latency across decode and prefill sweeps.

C. Heuristic Policy

The repository also defines a simple heuristic for selecting the split-K factor:

$$\text{split_k}(M) = \begin{cases} 4, & M \leq 32, \\ 2, & 32 < M \leq 128, \\ 1, & M > 128. \end{cases}$$

In the plots, this policy appears as the `triton heuristic` curve. In practice, however, the heuristic largely mirrors the explicitly benchmarked split-K settings. In the decode regime, it coincides with the split-K $= 4$ configuration for all tested shapes. In the prefill regime, it selects split-K $= 2$ only at $M = 128$, and otherwise reduces to split-K $= 1$. Consequently, the heuristic should be understood primarily as a simple selection rule over the existing kernel variants rather than as an independently optimized scheduling strategy.

III. Experimental Setup

The benchmark harness fixes the reduction dimension at $K = 8192$ and sweeps over

$$N \in \{4096, 8192, 11008\}.$$

For each shape, the harness generates random FP16 activation matrices and random FP16 weight matrices, quantizes the weights to INT4, and then measures runtime using `triton.testing.do_bench`. The default configuration uses a warmup period of 100 ms, followed by a 200 ms measurement window, and reports the mean latency in milliseconds.

All measurements are written to a CSV file that records latency for each method, inference regime, and matrix shape. The plotting script then produces three multi-panel figures summarizing the results. The first reports absolute latency. The second reports speedup relative to the naive dequantize-then-matmul baseline. The third reports speedup relative to the fused data-parallel baseline. Together, these views provide both an absolute and a relative assessment of the trade-offs among the compared implementations.

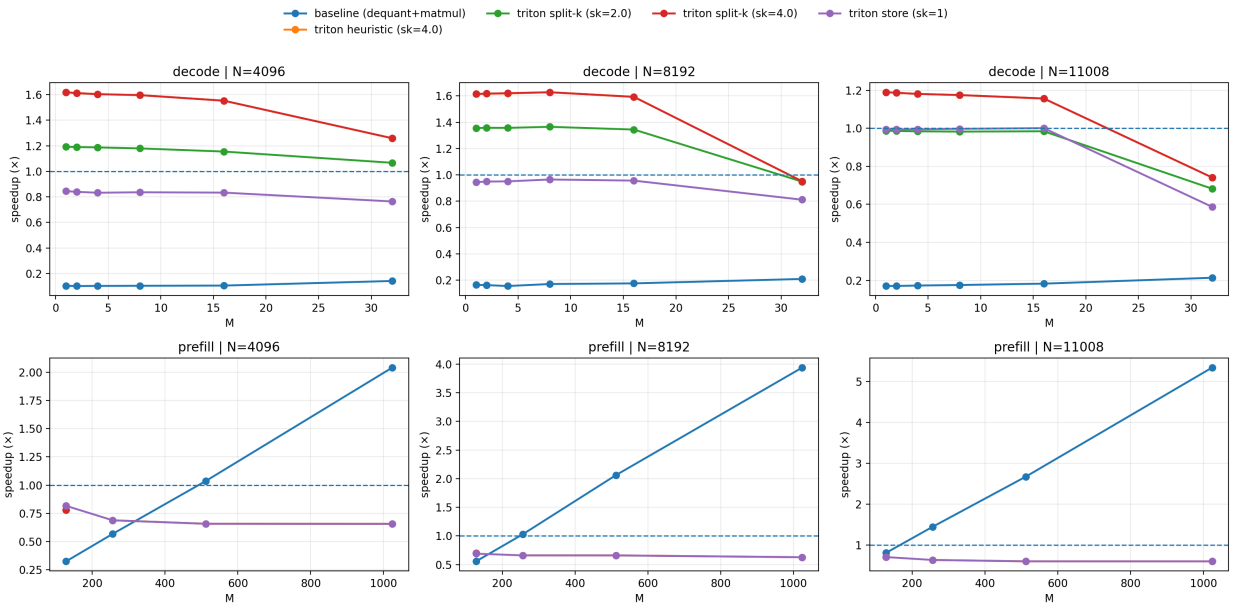

Fig. 2: Speedup relative to the fused data-parallel baseline. Values above 1 indicate improvement over the paper-style baseline.

IV. Results

A. Overall Latency Trends

Figure 1 shows the absolute latency grid. Three observations are immediate.

First, the naive baseline is almost flat as a function of M for every N. This suggests that, in the current implementation, repeatedly dequantizing the full weight matrix is a large fixed cost and dominates much of the runtime until M becomes very large.

Second, the fused Triton kernels dramatically reduce latency in decode. At $N = 4096$, the best decode latency is approximately 0.38–0.41 ms for $M \leq 16$, compared with about 6.0 ms for the naive baseline. At $N = 8192$, the best decode latency is about 0.61–0.67 ms for $M \leq 16$. At $N = 11008$, the best decode latency rises to about 0.86–0.94 ms, but still substantially outperforms the naive baseline.

Third, the prefill picture is different. The fused data-parallel baseline remains the fastest Triton kernel, but all custom split-K or store-based variants scale poorly with M. By $M = 512$ and $M = 1024$, even the fused data-parallel kernel is slower than the naive baseline for all tested N.

B. Decode: Split-K Helps, but Only in the Small-M Regime

Figure 2 is the most informative for the decode question because it normalizes all methods against the fused data-parallel baseline. The main result is that split-K $= 4$ is the best decode setting in almost every small-M case:

- At $N = 4096$, split-K $= 4$ beats the fused data-parallel baseline by about $1.55\times$ to $1.62\times$ for $M \leq 16$, and still wins by $1.26\times$ at $M = 32$.
- At $N = 8192$, split-K $= 4$ beats the fused data-parallel baseline by about $1.59\times$ to $1.63\times$ for $M \leq 16$, but the advantage disappears at $M = 32$, where the data-parallel baseline becomes slightly faster.
- At $N = 11008$, split-K $= 4$ still wins for $M \leq 16$, but only by about $1.16\times$ to $1.19\times$. At $M = 32$, the fused data-parallel baseline is clearly faster.

These trends are consistent with the intended role of split-K. When M is tiny, the output tile grid has limited parallelism, so splitting the reduction dimension creates more independent work. As M or N grows, ordinary data-parallel tiling already exposes enough concurrency, and the extra writeback plus reduction overhead of the split-K path becomes increasingly difficult to justify.

Two negative results are also useful:

- The explicit split-K $= 2$ setting is never the best-performing point in the measured sweep.
- The fixed-tile store kernel with split-K $= 1$ never beats the autotuned fused data-parallel baseline.

This means that the custom implementation currently has a very specific sweet spot: split-K $= 4$ for decode, not split-K in general.

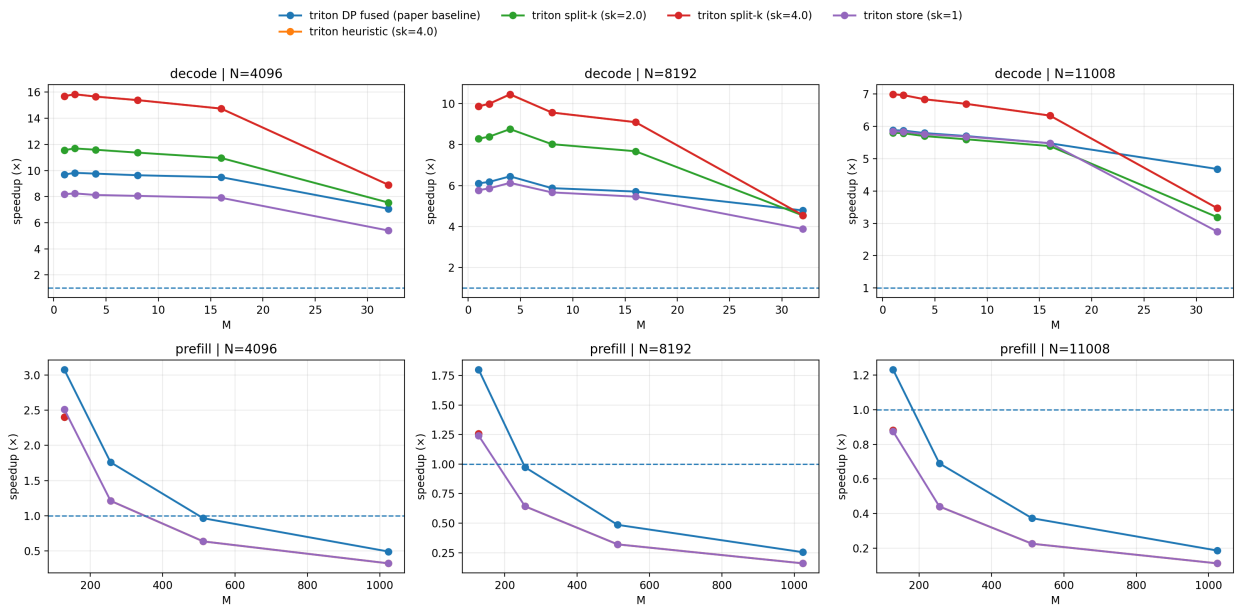

Fig. 3: Speedup relative to the naive dequantize-then-matmul baseline. Values above 1 indicate improvement over the unfused baseline.

C. Speedup Relative to the Naive Baseline

Figure 3 shows the speedup of each fused kernel relative to the dequantize-then-matmul baseline.

In decode, the gains are large. The best measured decode speedups over the naive baseline are:

- $15.7\times$ at $(M, N) = (1, 4096)$ with split-K $= 4$;
- $10.5\times$ at $(M, N) = (4, 8192)$ with split-K $= 4$;
- about $7.0\times$ at $(M, N) = (1, 11008)$ with split-K $= 4$.

Even when $M = 32$, the best fused implementation still yields about $8.9\times$ speedup at $N = 4096$ and about $4.7\times$–$4.8\times$ at the larger N values.

In prefill, the story reverses as M grows. At $M = 128$, the autotuned fused data-parallel kernel is still ahead of the naive baseline by $3.08\times$ for $N = 4096$, $1.80\times$ for $N = 8192$, and $1.23\times$ for $N = 11008$. After that, the crossover happens quickly:

- at $N = 4096$, the fused data-parallel kernel is still faster at $M = 256$, but not at $M \geq 512$;
- at $N = 8192$ and $N = 11008$, the naive baseline is already competitive by $M = 256$ and clearly faster by $M \geq 512$.

This suggests that the current Triton fused kernels are well matched to decode, but do not yet deliver an equally strong prefill kernel for larger matrix heights.

V. Interpretation

Taken together, the results support three conclusions.

a) The core contribution this quarter is a decode-oriented result.: The no-atomic split-K design is not universally better than a data-parallel fused kernel, but it is meaningfully better when M is small. This is exactly the regime that matters for autoregressive decode, where batch-size-like dimensions are often too small to fully utilize the device with a purely data-parallel mapping.

b) The current heuristic is useful, but not yet shape-aware enough.: The hard-coded policy succeeds in decode because it mostly maps directly to split-K $= 4$, which is the best measured choice there. However, it is not robust outside that regime. At prefill $M = 128$, the heuristic selects split-K $= 2$, yet the autotuned data-parallel baseline is faster. At decode $M = 32$ and larger N, the heuristic still selects split-K $= 4$ even though the data-parallel baseline has already crossed over and become better. A better policy should depend on both M and N, and possibly on an explicit cost model for partial-buffer traffic.

c) The benchmark also highlights engineering gaps rather than only algorithmic effects.: The fused data-parallel baseline is autotuned, while the custom store and split-K kernels currently use fixed tile sizes. Therefore, part of the observed gap between the custom kernels and the data-parallel baseline is likely due to tuning quality rather than to the underlying algorithm alone. This does not invalidate the decode result, because the split-K $= 4$ path still wins despite the extra reduction step, but it does mean the prefill comparison should be interpreted as "current implementation quality" rather than as a final

statement about what Triton can do.

VI. Limitations

First, the compared methods were not tuned with the same level of effort. The fused data-parallel baseline benefits from Triton's autotuning mechanism, whereas the custom kernels rely on fixed meta-parameters chosen manually. This asymmetry means that part of the observed performance gap may reflect differences in tuning methodology rather than purely differences in algorithmic structure.

Second, correctness validation is still limited in scope. The repository includes a correctness test for the custom Triton implementation against the naive baseline on a single small problem shape, but the fused data-parallel baseline and the full benchmark sweep have not yet been covered by a broader regression-testing framework. The reported performance results should therefore be understood in the context of incomplete end-to-end validation across all tested configurations.

Finally, the semantics of the naive baseline should be interpreted carefully. Because it dequantizes the weights on every invocation before performing matrix multiplication, it represents an unfused end-to-end reference rather than a fully optimized large-M baseline. Accordingly, speedups relative to this baseline capture the advantage of fusion together with the avoidance of repeated dequantization overhead.

VII. Next Steps

The current results suggest several priorities for the next stage of work.

First, the custom store and split-K kernels should be autotuned rather than evaluated with a single fixed tile configuration. This would allow a fairer comparison with the fused data-parallel baseline and clarify whether the current gaps are due to kernel design or to limited tuning.

Second, the hard-coded split-K heuristic should be replaced with a more shape-aware policy. The present rule depends only on M, but the best choice is likely influenced by both M and N, especially across decode and prefill regimes.

Finally, it may be worthwhile to design a separate fused kernel for prefill. The current implementation appears most effective in decode, but less competitive when M is large.

VIII. Conclusion

This quarter's work produced a functioning W4A16 Triton benchmark framework, a fused paper-style data-parallel baseline, and a custom split-K implementation that avoids atomics through partial accumulation and reduction. The most important empirical result is that split-K $= 4$ is decisively beneficial in the decode regime, giving up to about $1.6\times$ speedup over the fused data-parallel baseline and up to about $15.7\times$ speedup over the naive dequantize-then-matmul implementation. However, the benefit is highly regime-dependent: as the shape moves toward prefill, the extra reduction overhead outweighs the concurrency advantage, and the autotuned data-parallel baseline becomes the best Triton variant. The current code therefore demonstrates a strong decode result and a clear roadmap for improving shape awareness, tuning quality, and prefill performance.